

Lab 10: Intel 8254 PIT (Mode 2)

MUP Lab 10

Overview

This week, you will explore the architecture of the **Intel 8254 Programmable Interval Timer (PIT)**. Rather than just writing software drivers, you will implement the actual internal hardware logic using Verilog HDL. By the end of this lab, you will have built a functional 8254 module tailored to **Mode 2 (Rate Generator)** operation.

1 Module Descriptions and File Tasks

To help you understand the architecture, the 8254 is broken down into modular files. Some are provided for you, while others require your implementation. Note that everything has been set to POSEDGE clk.

1. Data Bus Buffer (*Provided*)

- **Role:** Passes the external bidirectional data bus into the internal chip modules securely.
- **Input:** `data_in[7:0]`
- **Output:** `data_out[7:0]`

2. Read/Write Logic (*Provided*)

- **Role:** Decodes the address lines (A_1, A_0) and generates individual write-enable signals for the counters and control register.
- **Inputs:** `cs`, `wr`, `a1`, `a0`
- **Outputs:** `ctr0_wr`, `ctr1_wr`, `ctr2_wr`, `ctrl_reg_wr`

3. Control Register (*To Implement*)

- **Role:** Accepts an 8-bit control word and permanently stores it into the correct counter's configuration register.
- **Task:** Check bits [7:6] of the incoming `data_in`. Route the 8-bit word into `ctr0_control`, `ctr1_control`, or `ctr2_control` based on the table below:

SC1 (Bit 7)	SC0 (Bit 6)	Destination Register
0	0	Select Counter 0
0	1	Select Counter 1
1	0	Select Counter 2
1	1	<i>Read-Back Command(Ignore/Retain previous in this lab)</i>

4. Counter (*To Implement*)

- **Role:** The core counting logic. It decrements its count value on every clock cycle and toggles the OUT pin appropriately.

Internal Variables:

- **count:** The active 16-bit register that decrements every clock cycle.
- **reload_value:** Stores the initial value loaded from the data bus. Used to auto-reload the count when it hits 1.
- **byte_state:** A 1-bit flag used *only* in 16-bit load mode (`rw_mode == 2'b11`) to track if the incoming byte is the LSB (0) or the MSB (1).

Actionable Tasks:

- **Initialization (initial block):** Set count, reload_value, and byte_state to 0. Set out to 1 default idle state.
- **Load Logic (ctr_wr):**
 - * For `rw_mode == 2'b01` (LSB), `2'b10` (MSB), assign data_in to the relevant 8 bits of count and reload_value, set rest of the bits accordingly.
 - * For `rw_mode == 2'b11` (16-bit), check byte_state to switch between MSB, LSB.
- **Counting Logic (valid_mode):**
 - * **If gate is HIGH:** Refer to Mode 2: Rate Generator & Gate Logic section in 2 Theory and Architecture in this pdf.

5. Top Level Module (*To Implement*)

- **Role:** Connects the Data Bus Buffer, R/W Logic, Control Register, and all 3 Counters together.
- **Address Map (A_1, A_0):** The address lines dictate which target is being accessed during a write operation:
 - 00: Counter 0
 - 01: Counter 1
 - 10: Counter 2
 - 11: Control Register
- **Design Simplifications:**
 - **Active-High Logic:** The physical 8254 chip utilizes active-low control signals (e.g., $\overline{CS}$, $\overline{WR}$). For the sake of simplicity in this hardware modeling lab, all logic is strictly positive and active-high, operating entirely on `posedge clk`.

- **Ignoring the Read Signal (rd):** The `rd` signal is present in the port list to match the architecture, but you may safely ignore it.
- **Tasks:** Instantiate modules and Wire the `int_data_out` bus, the individual write-enable wires (`c0_wr`, etc.), and the control buses (`c0_ctrl`, etc.) together.

2 Theory and Architecture

Module Connections

The 8254 relies on an internal bus system. The **Data Bus Buffer** safely isolates the external `data_in` and feeds it as an internal bus to both the **Control Register** and the **Counters**. Simultaneously, the **Read/Write Logic** acts as the brain’s decoder—it takes the chip select (`CS`), write enable (`WR`), and address lines (`A1`, `A0`) to activate exactly one internal component’s write-enable line at a time.

Tip: Refer to the provided `Mup_lab_10_Diagram.pdf` to visually map out these wire assignments before writing your `top.v` instantiations.

The Control Word

Before a counter can operate, it must be configured via the Control Register using an 8-bit word format:

- **Bits [7:6] (Select Counter):** 00 = Counter 0, 01 = Counter 1, 10 = Counter 2.
- **Bits [5:4] (Read/Write Mode):** 01 = LSB only, 10 = MSB only, 11 = LSB first, then MSB. (You will use the `byte_state` flip-flop to track this two-step process).
- **Bits [3:1] (Mode):** For this lab, valid Mode 2 values are 010 or 110.
- **Bit [0] (BCD):** 0 = Binary counter 16-bits (Ignore in this lab).

Mode 2: Rate Generator & Gate Logic

Mode 2 functions as a divide-by-N periodic counter. The `GATE` pin acts as a hardware counting enabler:

- **Gate HIGH (1):** Normal counting operations occur. The `OUT` signal remains **HIGH** during normal deceleration. When the count hits exactly **1**, `OUT` goes **LOW** for exactly one clock cycle. The counter immediately auto-reloads its initial value on the next clock and the cycle repeats.
- **Gate LOW (0):** Counting is suspended. The `OUT` signal remains **HIGH**.

Example Timing Diagram (Count = 4, Gate = 1)

Clock Cycle	1	2	3	4	5	6	7	8	9
Count Value	4	3	2	1	4	3	2	1	4
OUT Signal	H	H	H	L	H	H	H	L	H

3 Marking Scheme (Total: 15 Marks)

To facilitate partial marks, your grade is divided by passing each testcase (*0.5 marks per testcase*).

Category	Criteria / Testcases	Marks
control_register.v	4 Testcases: Correct routing of control words & Invalid SC	2.0
counter.v	3 Testcases: Basic countdown and auto-reload	1.5
	3 Testcases: Gating, count=1 safety bounds & Invalid Mode	1.5
	2 Testcases: MSB/LSB 16-bit load combinations	1.0
top.v	4 Testcases: Basic counter integration & Gate disable	2.0
	4 Testcases: Multi-counter edge cases & 16-bit countdown	2.0
Attendance	Lab session attendance	2.5
Submission	Successful submission	2.5
Total		15.0

Note: Testbenches have been provided in your starter folder (`tb_control_register.v`, `tb_counter.v`, and `tb.v`). You may use these or directly use the python file, and grading will be done by the same test cases with our own testbenches.

4 Testing Workflow

Run All Tests

```
python3 run_tests.py
```

Run a Single Module Test

```
python3 run_tests.py counter
python3 run_tests.py control_register
python3 run_tests.py top
```

5 Submission

After completing all modules and verifying your outputs, run the grading script:

```
python3 create_submission.py
```

When prompted, enter your ID in the standard format (e.g., 2024A7PS9999G). This will safely bundle your `.v` source files into a ZIP format. Upload that ZIP file directly to the lab portal.